Operating Systems and Systems Programming (25CS2104E)

Inter-Process Chat Application Using IPC

Team Members

Varalakshmi K – 2520030678

Lohitha M – 2520030378

Vaibhavasree S – 2520030398

Under the Esteemed Guidance of

K. Anusha, Faculty, Dept. of CSE

Contents

- Abstract
- Problem Statement
- Objectives
- Proposed Methodology & OS Concepts Used
- Flow Chart
- Individual Contribution
- Tools & Platforms Used
- Expected Outcome
- GitHub Repository & Conclusion

Abstract

The Inter-Process Chat Application Using IPC is a Linux-based Operating Systems project in which separate processes running on the same machine exchange chat messages through Inter-Process Communication (IPC) mechanisms provided by the OS. Since every process has its own private memory space, processes cannot read each other's data directly, so OS-level IPC is required for them to communicate.

The proposed system uses **named pipes and Unix domain sockets**. Named pipes (FIFO) support a two-user chat, while a central server process with Unix domain sockets supports a multi-user group chat by accepting client connections and broadcasting each message to the other clients. Threads allow sending and receiving at the same time, and a lock protects shared data.

The project demonstrates Operating Systems concepts such as processes, IPC, named pipes, sockets, multithreading, synchronization, blocking I/O, and file descriptors. It is implemented in Python on Linux/Ubuntu and operated through the terminal, where each terminal window acts as a separate chat user.

Problem Statement

- Every process in an operating system runs in its own private memory space, so one process cannot directly read or write another process's data.
- Separate processes, such as different chat users, therefore need an OS-provided mechanism to exchange messages and coordinate with each other.
- There is a need for a simple chat system that shows how processes on one machine communicate in real time without using a network.
- The system uses named pipes for a two-user chat and Unix domain sockets with a server process for a multi-user group chat.
- Simultaneous sending and receiving, and access to shared data, must be handled using threads and locks to avoid blocking and race conditions.
- This is relevant to Operating Systems as it demonstrates processes, IPC mechanisms, synchronization, file descriptors, and blocking I/O.

Objectives

- To build a chat application in which separate processes communicate using OS-level IPC.
- To implement a two-user chat using named pipes (FIFO).
- To implement a multi-user group chat using a server process and Unix domain sockets.
- To handle simultaneous sending and receiving using threads, and shared data using locks.
- To support join/leave notices and a clean shutdown that removes pipe and socket files.
- To compare IPC mechanisms (pipes, message queues, shared memory, sockets) and demonstrate practical OS concepts.

Methodology & OS Concepts

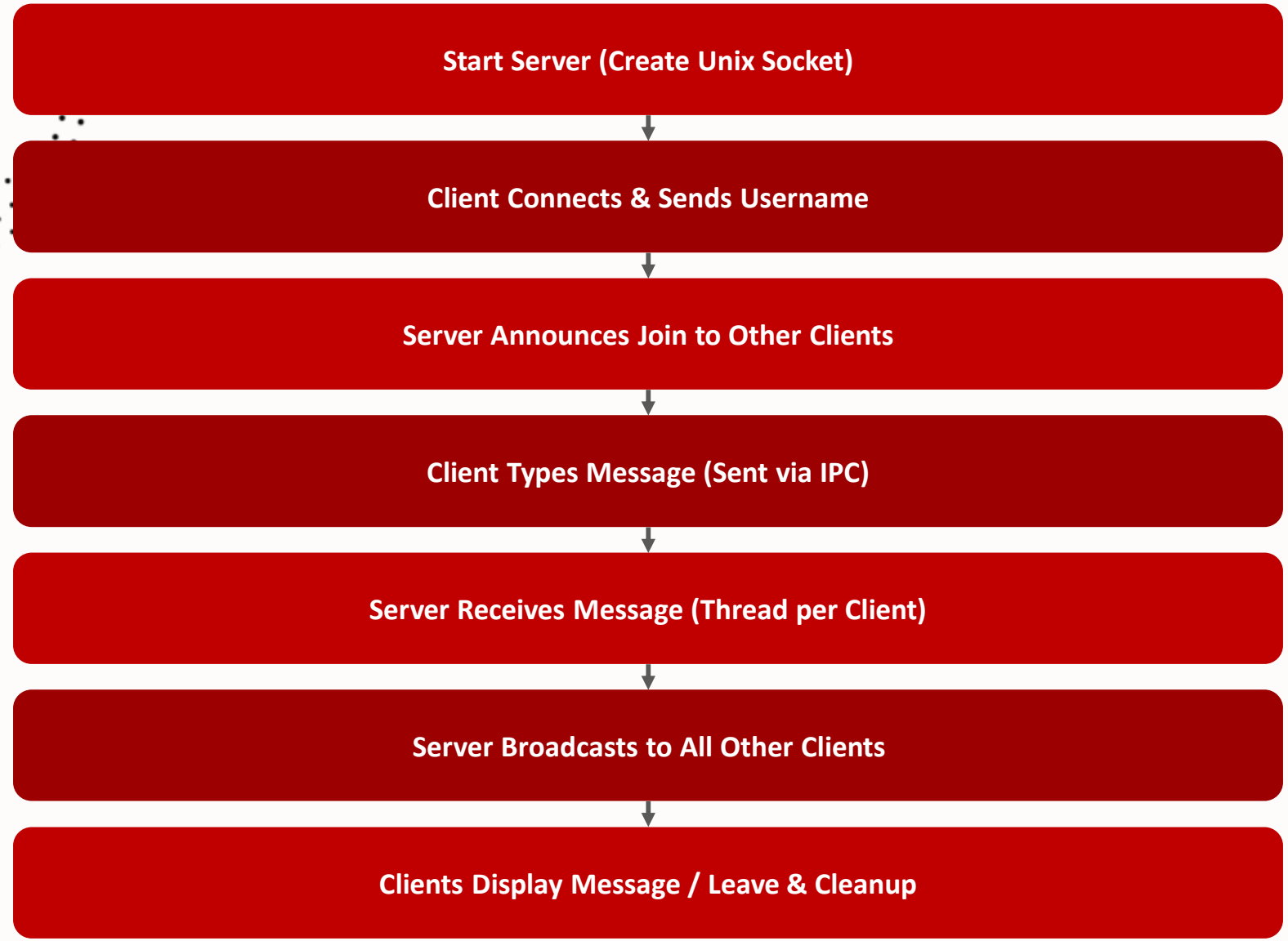
Proposed Methodology

Built in Python on Linux/Ubuntu using OS-level IPC. Version 1 uses two named pipes for a two-user chat. Version 2 runs a server on a Unix domain socket; clients connect and send a name, and the server broadcasts every message to the other clients, using threads for concurrent I/O and a lock for the client list. Flow: Start Server → Client Connects → Send Message → Server Receives → Broadcast → Clients Display.

OS Concepts / IPC APIs Used

- os.mkfifo() – creates a named pipe (FIFO) as an IPC channel.
- socket(AF_UNIX) / bind() / listen() – sets up the local server socket.
- accept() / connect() – server accepts clients; clients join the chat.
- send() / recv() – exchange messages between processes.
- threading.Thread / Lock – concurrent I/O and synchronization.
- close() / os.remove() – release sockets and delete pipe files on exit.

Flow Chart


Start Server (Create Unix Socket)
Client Connects & Sends Username
Server Announces Join to Other Clients
Client Types Message (Sent via IPC)
Server Receives Message (Thread per Client)
Server Broadcasts to All Other Clients
Clients Display Message / Leave & Cleanup

Individual Contribution

- **Varalakshmi K (2520030678):** Client program, multithreaded send/receive handling, and user input interface.
- **Lohitha M (2520030378):** Project architecture, server program, Unix domain socket setup, and message broadcast logic.
- **Vaibhavasree S (2520030398):** Named-pipe two-user chat, synchronization, testing, debugging, documentation, README, and final demonstration.

Tools / Platforms / Software Used

Operating System & Language

- Linux / Ubuntu – operating system and execution environment
- Python 3 – main programming language
- Python modules – os, socket, threading, sys

APIs & Development Tools

- Named pipes (FIFO) – two-user IPC channel
- Unix domain sockets – multi-user IPC channel
- VS Code / Nano – source-code development
- Git – version control
- GitHub – source-code repository and project submission

Expected Outcome

The completed system is expected to let multiple separate processes chat with each other on one Ubuntu machine through OS-level IPC. Two terminals will exchange messages in real time using named pipes, and any number of clients will exchange messages through the server using Unix domain sockets, with join/leave notices and a clean shutdown. The project demonstrates practical knowledge of processes, IPC mechanisms, sockets, multithreading, synchronization, and file descriptors — delivering a lightweight terminal-based chat application that clearly shows how processes communicate inside an operating system.

Conclusion

- GitHub Repository: github.com/muvvalohitha007-cmyk/TRIDE_OSSP
- Section No.: 06 | Team No.: 08 | Faculty Guide: K. Anusha, Dept. of CSE
- The project demonstrates practical Operating Systems concepts: processes, IPC, named pipes, sockets, threads, and synchronization.
- It delivers a lightweight, real-time chat application in which separate processes communicate through OS-provided channels.
- The solution correctly handles simultaneous messaging, user join/leave events, and clean removal of pipe and socket files.
- Future enhancements may include private messages, chat logs, message-queue or shared-memory versions, TCP sockets for network chat, and a GUI.

(DEEMED TO BE UNIVERSITY)

Thank you